Universidad de Ingeniería y Tecnología (UTEC)
CS3501 – Arquitectura de Computadoras

Proyecto 2: Procesador RISC-V superescalar Dual-Issue

Informe de la primera entrega (E1)

Pipeline RV32I con Hazard Unit y arquitectura Dual-Issue preliminar

Integrantes

Kenzo Akira Shibuya Rengifo
Gabriel Marcelo Diaz Carrion
Brisseth Surquislla Gamarra

Índice

1. Introducción **2**

2. Pipeline de cinco etapas, datapath y unidad de control **2**
2.1. Etapas . . . 2
2.2. Datapath . . . 2
2.3. Unidad de control . . . 3

3. Hazard Unit y datapath Single-Issue **3**
3.1. Cambios realizados respecto a la versión inicial . . . 4
3.2. Diagrama del datapath Single-Issue . . . 4

4. Programas de prueba y verificación **5**
4.1. Metodología . . . 5
4.2. Programas . . . 5
4.3. Resultados en memoria . . . 5

5. Forwarding, stalling y flushing observados **6**
5.1. Programa 1: sin intervención de la Hazard Unit . . . 6
5.2. Programa 2: forwarding . . . 8
5.3. Programa 3: stalling . . . 10
5.4. Programa 4: flushing . . . 12

6. Extensión Dual-Issue: Fetch, Decode y datapath preliminar **13**
6.1. Cambios en Fetch . . . 13
6.2. Cambios en Decode . . . 13
6.3. Módulos incorporados . . . 14
6.4. Datapath Dual-Issue preliminar . . . 14
6.5. Evidencia de Dual Fetch y Dual Decode . . . 15

7. Conclusiones **16**

1. Introducción

El objetivo del Proyecto 2 es diseñar un procesador RISC-V RV32I superescalar *Dual-Issue In-Order* a partir de un pipeline de cinco etapas. En esta primera entrega se presenta el pipeline RV32I funcional, con *forwarding*, *stalling* y *flushing*, y el inicio de la arquitectura superescalar: Dual Fetch, Dual Decode e incorporación de los módulos de la segunda vía. Todos los resultados de este informe provienen de la simulación del diseño en Verilog.

2. Pipeline de cinco etapas, datapath y unidad de control

El procesador base es un RISC-V RV32I *pipelined* de cinco etapas (IF, ID, EX, MEM, WB), derivado del diseño single-cycle de Harris & Harris. Cada etapa procesa una instrucción distinta en el mismo ciclo, por lo que en régimen estacionario se completa una instrucción por ciclo (CPI ideal = 1). El periodo de reloj queda limitado por la etapa más lenta y no por la suma de todas.

2.1. Etapas

Etapa	Función	Módulos principales	Registro de salida
IF (Fetch)	Lee la instrucción en `PCF` y calcula `PCPlus4F`. Elige el siguiente PC entre `PCPlus4F` y `PCTargetE`.	`pcreg` (`flopenr`), `pcadd4`, `pcmux`, `imem`	IF/ID (`flopenrc`): `InstrD`, `PCD`, `PCPlus4D`
ID (Decode)	Decodifica la instrucción, lee `rs1`/`rs2` del banco de registros y extiende el inmediato.	`controller` (`maindec` + `aludec`), `regfile`, `extend`	ID/EX (`floprc`): control, `RD1E`, `RD2E`, `ImmExtE`, `PCE`, `Rs1E`, `Rs2E`, `RdE`
EX (Execute)	Elige los operandos (con *forwarding*), opera en la ALU, calcula el destino de salto y resuelve `beq`/`jal`.	`forwarda_mux`, `forwardb_mux`, `srcbmux`, `alu`, `pcaddbranch`	EX/MEM (`flopr`): `ALUResultM`, `WriteDataM`, `RdM`
MEM (Memory)	Accede a la memoria de datos: escribe en `sw` y lee en `lw`.	`dmem` (externa al núcleo)	MEM/WB (`flopr`): `ALUResultW`, `ReadDataW`, `PCPlus4W`
WB (Write-back)	Elige el resultado (ALU, memoria o PC+4) y lo escribe en el banco de registros.	`resultmux`, `regfile` (puerto de escritura)	—

Tabla 1: Etapas del pipeline y módulos que las implementan.

Las señales llevan el sufijo de la etapa en que se usan (`InstrD`, `ALUResultM`, `RdW`), lo que permite seguir cada instrucción a lo largo de los registros de pipeline.

2.2. Datapath

Respecto al diseño *single-cycle*, el datapath agrega cuatro registros de pipeline que transportan datos y control de una etapa a la siguiente:

- **IF/ID** usa `flopenrc` (enable y clear): el enable `~StallD` congela la etapa y el clear `FlushD` la convierte en burbuja (hazard bubble).
- **ID/EX** usa `floprc` (clear): `FlushE` inserta una burbuja en EX.
- **EX/MEM** y **MEM/WB** usan `flopr`: nunca se detienen ni se vacían.
- El **PC** usa `flopenr` con enable `~StallF`.

El destino del salto ($PCTargetE = PCE + ImmExtE$) y la decisión $PCSrcE = (BranchE \cdot ZeroE) + JumpE$ se calculan en EX. Por ello, un salto tomado obliga a descartar las dos instrucciones que ya ingresaron a IF e ID. El banco de registros escribe en el **flanco de bajada** (negedge) y lee combinacionalmente: una instrucción en WB y otra en ID que acceden al mismo registro en el mismo ciclo no generan riesgo, porque la escritura ocurre en la primera mitad del ciclo y la lectura en la segunda.

2.3. Unidad de control

La unidad de control es combinacional y opera en ID. Se compone de dos decodificadores:

- `maindec` decodifica el *opcode* y genera `RegWrite`, `ImmSrc`, `ALUSrc`, `MemWrite`, `ResultSrc`, `Branch`, `ALUOp` y `Jump`.
- `aludec` combina `ALUOp`, `funct3`, `funct7[5]` y `op[5]` para generar `ALUControl[2:0]`.

Las señales de control viajan por los registros de pipeline junto con la instrucción y se consumen en su etapa: `ALUSrc` y `ALUControl` en EX, `MemWrite` en MEM, y `ResultSrc` y `RegWrite` en WB.

Instrucción	opcode	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp	Jump
lw	0000011	1	00	1	0	01	0	00	0
sw	0100011	0	01	1	1	00	0	00	0
R-type	0110011	1	xx	0	0	00	0	10	0
beq	1100011	0	10	0	0	00	1	01	0
I-type ALU	0010011	1	00	1	0	00	0	10	0
jal	1101111	1	11	0	0	10	0	00	1
default	0000000	0	00	0	0	00	0	00	0

Tabla 2: Señales generadas por `maindec`.

ISA soportada: `add`, `sub`, `and`, `or`, `slt`, `addi`, `andi`, `ori`, `slti`, `lw`, `sw`, `beq` y `jal`. La ALU incluye los códigos de `sll`/`srl`, pero `aludec` aún no los decodifica.

3. Hazard Unit y datapath Single-Issue

La Hazard Unit (`hazard.v`) es combinacional. Resuelve los riesgos de datos mediante *forwarding* y el *stall* por Load-Use, y los riesgos de control mediante *flush*. La Tabla 3 resume las señales que genera.

Señal	Condición	Efecto
`ForwardAE = 10`	`Rs1E == RdM`, `RegWriteM` y `Rs1E != 0`	El operando A toma `ALUResultM` (resultado de la instrucción anterior).
`ForwardAE = 01`	`Rs1E == RdW`, `RegWriteW` y `Rs1E != 0` (si no se cumple la anterior)	El operando A toma `ResultW` (resultado de hace dos instrucciones).
`ForwardBE = 10/01`	Las mismas condiciones con `Rs2E`	Afecta a `SrcBE` y también a `WriteDataE` (dato de un `sw`).
`lwStall`	`ResultSrcE[0]` (hay un `lw` en EX), `RdE != 0` y (`Rs1D == RdE` o `Rs2D == RdE`)	`StallF = StallD = 1` y `FlushE = 1`: se repite la instrucción en ID y se inserta una burbuja en EX.
`PCSrcE`	`beq` tomado o `jal` en EX	`FlushD = FlushE = 1`: se anulan las dos instrucciones del camino incorrecto.

Tabla 3: Señales de la Hazard Unit.

La prioridad **MEM sobre WB** garantiza que, si dos instrucciones seguidas escriben el mismo registro, se reenvíe el valor más reciente. Un `lw` exige un ciclo de espera porque su dato recién existe

al final de MEM, mientras que el consumidor lo necesita al inicio de EX; tras ese ciclo, el dato llega por `ResultW` (`Forward = 01`).

3.1. Cambios realizados respecto a la versión inicial

1. **Burbuja válida en `maindec`.** Tras un reset o un *flush*, `InstrD` vale `0x00000000` (opcode `0000000`). La versión inicial no tenía ese caso y caía en el `default`, que asigna `x` a todas las señales de control. En simulación, el primer `beq` tomado propagaba `x` a `JumpE`/`BranchE`, luego a `PCSrcE` y finalmente al PC (`PCF = xxxxxxxx`), y el programa nunca terminaba. Se agregó el caso `7'b0000000: controls = 0`, de modo que la burbuja, o la instrucción nop en este caso, no escribe registros ni memoria y no salta.

2. **`lwStall` ignora `x0`.** Se agregó la condición `RdE != 0`. Un `lw x0, ...` ya no detiene el pipeline cuando la siguiente instrucción lee `x0` (ver Programa 3).

3. **Registros de pipeline portables.** `flopr`, `flopenr`, `floprc` y `flopenrc` declaraban `output [..] q` y luego `reg q`. Icarus Verilog rechaza esa redeclaración, por lo que se reescribieron con `#(parameter WIDTH)` y `output reg`. El comportamiento no cambia.

Observación. La unidad compara `Rs2` aunque la instrucción sea de tipo I. En ese caso esos bits pertenecen al inmediato y pueden aparecer `ForwardBE != 00` sin efecto, porque `ALUSrcE = 1` selecciona el inmediato. Por ejemplo, en el Programa 2, `addi x7, x6, 5` muestra `ForwardBE = 01`.

3.2. Diagrama del datapath Single-Issue

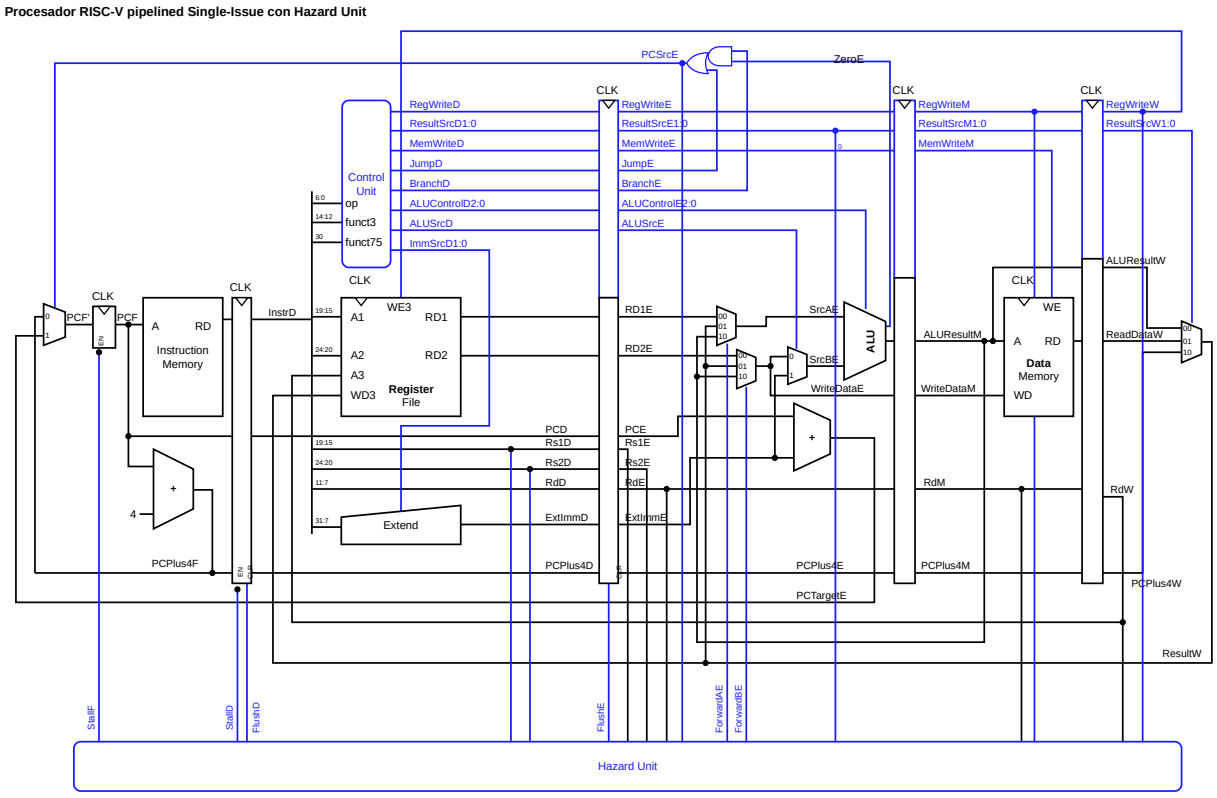


Figura 1: Datapath pipelined Single-Issue con Hazard Unit.

La Figura 1 muestra el datapath completo. Las señales de control (azul) se generan en la unidad de control y avanzan por la parte superior de los registros de pipeline. La Hazard Unit recibe los identificadores de registros y las señales de escritura de cada etapa, y controla los enables y clears de los registros y los multiplexores de *forwarding*.

4. Programas de prueba y verificación

Cuatro programas en ensamblador RV32I verifican el pipeline, uno por cada mecanismo de la Hazard Unit. Los cuatro producen en memoria los mismos resultados que un simulador de referencia a nivel de instrucción, lo que confirma que el *forwarding*, los *stalls* y los *flushes* no alteran la semántica del programa.

4.1. Metodología

1. Cada programa se escribió en ensamblador (`.s`) y se ensambló a código máquina hexadecimal (`.mem`, una palabra por línea).
2. El archivo se carga mediante el parámetro `MEM_FILE` de `top`, de modo que el mismo diseño ejecuta los cuatro programas sin modificaciones.
3. Cada programa guarda sus resultados con `sw` a partir de la dirección 64 y termina en `done: beq x0, x0, done`. Los registros que deben quedar en 0 se inicializan explícitamente, porque el banco de registros no se resetea.
4. El testbench compara la memoria de datos final con los valores esperados. Durante la simulación se registran `ForwardAE`, `ForwardBE`, `StallF`, `FlushD` y `FlushE` en cada ciclo.

4.2. Programas

Programa	**Objetivo**	**Casos que cubre**
P1 `prog1_isa`	Ejecutar la ISA soportada sin que intervenga la Hazard Unit	`add`, `sub`, `and`, `or`, `slt`, `addi`, `andi`, `ori`, `slti`, `lw`, `sw` y `beq` no tomado; todo consumidor está a tres o más instrucciones de su productor.
P2 `prog2_forwarding`	Dependencias RAW resueltas solo con *forwarding*	MEM→EX y WB→EX en rs1 y rs2, prioridad MEM sobre WB, `x0` no se reenvía, dato de `sw` y operandos de `beq` reenviados.
P3 `prog3_stalling`	Dependencias Load-Use que exigen detener el pipeline	Load-Use en rs1, en rs2, en el dato de un `sw` y en un `beq`; casos sin *stall*: `lw` a distancia 2 y `lw x0`.
P4 `prog4_flushing`	Anulación de instrucciones ante cambios de flujo	`beq` tomado y no tomado, `jal` hacia adelante con registro de enlace y bucle con `jal` hacia atrás.

Tabla 4: Programas de prueba de la entrega E1.

En P4, cada instrucción que debe anularse suma 100, 200, 500, 600 o 1000 a un registro de resultado; si algún *flush* fallara, el error aparecería en la memoria final.

4.3. Resultados en memoria

Los valores de la Tabla 5 coinciden con el simulador de referencia en los cuatro programas. Un guion indica que el programa no escribe esa dirección.

Dirección	P1 ISA	P2 Forwarding	P3 Stalling	P4 Flushing
64	12	77	25	3
68	5	10	−7	44
72	17	13	25	6
76	7	3	50	0
80	240	15	32	–
84	252	1	−14	–
88	1	6	25	–
92	0	4	0	–
96	60	3	18	–
100	780	0	–	–
104	1	–	–	–
108	−5	–	–	–
112	12	–	–	–
116	5	–	–	–

Tabla 5: Contenido final de la memoria de datos.

5. Forwarding, stalling y flushing observados

Las tablas de esta sección son diagramas de pipeline obtenidos de la simulación. Muestran qué instrucción ocupa cada etapa en cada ciclo y qué señales activa la Hazard Unit; un guion representa una burbuja. El ciclo n abarca de $10n + 25$ a $10n + 35$ ns (reloj de 10 ns y reset hasta 22 ns), y la columna t indica el flanco de bajada en que se tomó la muestra. Las waveforms de cada programa muestran estos mismos eventos en esos tiempos.

5.1. Programa 1: sin intervención de la Hazard Unit

A lo largo de toda la fase de cómputo y almacenamiento de P1 (desde el fin del reset hasta que el último `sw` completa su etapa de Writeback), las señales `ForwardAE`, `ForwardBE`, `StallF`, `StallD`, `FlushD` y `FlushE` permanecen estrictamente en 0. La única activación de `Flush` ocurre recién al alcanzar el bucle de terminación `done: beq x0, x0, done` ($PC = 0$x90). Al evaluarse en EX, este salto siempre resulta tomado ($x_0 == x_0$), activando PCSrcE = 1 y generando FlushD = FlushE = 1 para descartar las dos instrucciones que el procesador ya había buscado especulativamente detrás del salto. Dado que el testbench verifica que el procesador cicla de forma estable esperando tres repeticiones en `done` antes de llamar a `$finish`, se observan exactamente tres pulsos de flush al cierre de la simulación, confirmando el correcto control del flujo final.

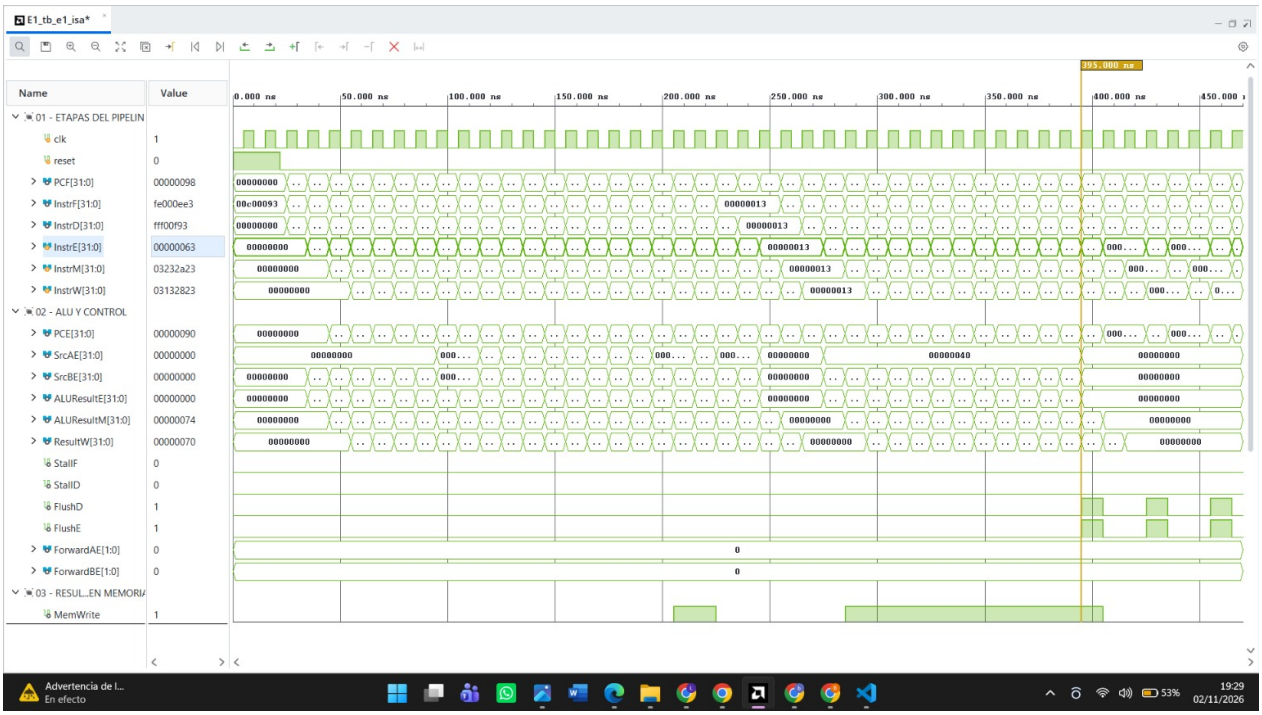


Figura 2: Waveform de P1: ForwardAE/BE y StallF en 0 durante todo el programa, y activación de FlushD/E únicamente en el bucle final done.

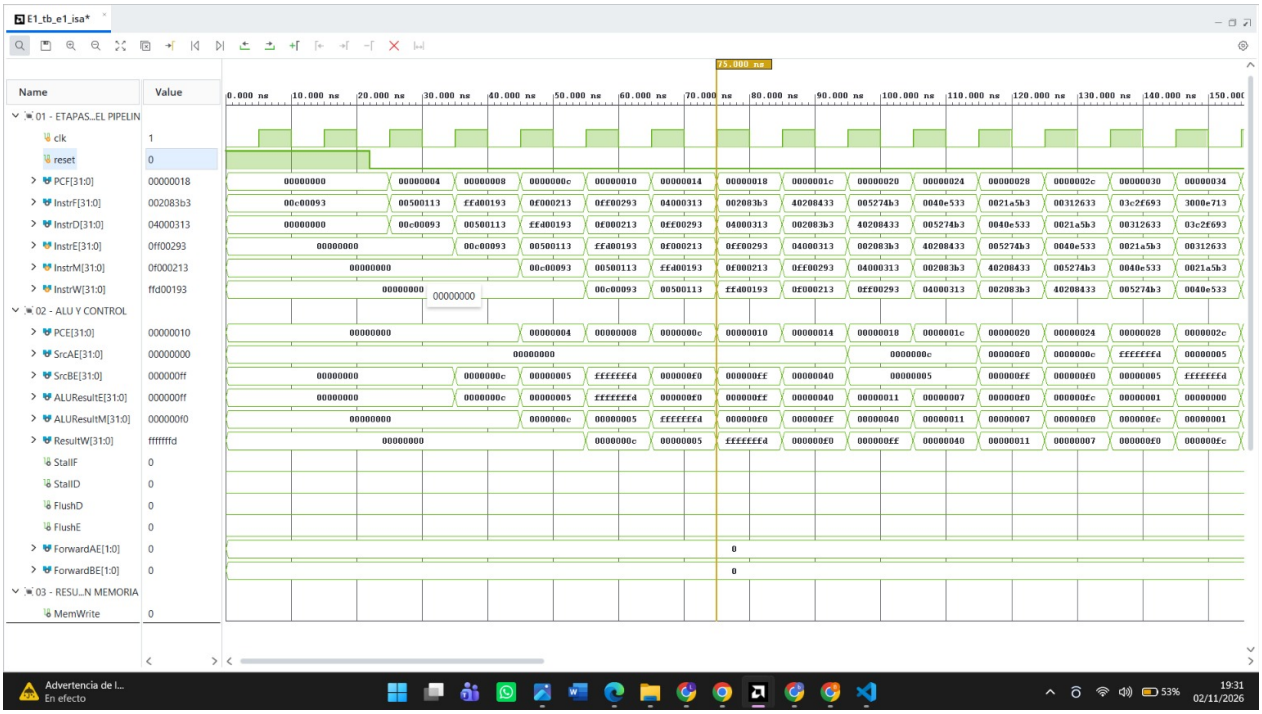


Figura 3: Waveform de P1: Avance de `PCF`, `InstrD`.

5.2. Programa 2: forwarding

Ciclo	t (ns)	IF	ID	EX	MEM	WB	Hazard Unit
5	80	`or x5, x2, x3`	`sub x4, x3, x1`	`add x3, x1, x2`	`addi x2, x0, 3`	`addi x1, x0, 10`	ForwardAE=01, ForwardBE=10
6	90	`and x6, x4, x3`	`or x5, x2, x3`	`sub x4, x3, x1`	`add x3, x1, x2`	`addi x2, x0, 3`	ForwardAE=10
7	100	`addi x7, x6, 5`	`and x6, x4, x3`	`or x5, x2, x3`	`sub x4, x3, x1`	`add x3, x1, x2`	ForwardBE=01
8	110	`addi x8, x0, 1`	`addi x7, x6, 5`	`and x6, x4, x3`	`or x5, x2, x3`	`sub x4, x3, x1`	ForwardAE=01
9	120	`addi x8, x0, 2`	`addi x8, x0, 1`	`addi x7, x6, 5`	`and x6, x4, x3`	`or x5, x2, x3`	ForwardAE=10, ForwardBE=01
12	150	`add x10, x0, x2`	`addi x0, x0, 7`	`add x9, x8, x8`	`addi x8, x0, 2`	`addi x8, x0, 1`	ForwardAE=10, ForwardBE=10
14	170	`sw x11, 0(x20)`	`addi x11, x0, 77`	`add x10, x0, x2`	`addi x0, x0, 7`	`add x9, x8, x8`	—
16	190	`nop`	`add x12, x9, x7`	`sw x11, 0(x20)`	`addi x11, x0, 77`	`add x10, x0, x2`	ForwardBE=10
19	220	`addi x14, x0, 5`	`addi x13, x0, 5`	`sw x12, 4(x20)`	`nop`	`add x12, x9, x7`	ForwardBE=01
22	250	`sw x3, 8(x20)`	`addi x15, x0, -1`	`beq x13, x14, ok`	`addi x14, x0, 5`	`addi x13, x0, 5`	ForwardAE=01, ForwardBE=10, FlushD, FlushE

Tabla 6: Diagrama de pipeline de P2 (ciclos seleccionados).

- **Ciclo 5:** `add x3, x1, x2` toma `x2` de MEM (`ALUResultM = 3`, `ForwardBE = 10`) y `x1` de WB (`ResultW = 10`, `ForwardAE = 01`).
- **Ciclos 6 a 9:** se alternan las cuatro combinaciones: A desde MEM, B desde WB, A desde WB y A desde MEM. En el ciclo 9, `addi x7, x6, 5` también muestra `ForwardBE = 01`, el *forwarding* sin efecto sobre el inmediato descrito en la Sección 3.
- **Ciclo 12 (prioridad):** ambas versiones de `x8` están en el pipeline, `x8 = 2` en MEM y `x8 = 1` en WB. La unidad elige MEM (`10`) para ambos operandos y `x9` vale 4.
- **Ciclo 14 (x0):** `addi x0, x0, 7` está en MEM con `RegWriteM = 1` y `RdM = 0`; aun así, `add x10, x0, x2` no recibe *forwarding*, porque la condición exige `Rs1E != 0`, y `x10` vale 3.
- **Ciclos 16 y 19 (store):** el dato del `sw` sale de `forwardb_mux` como `WriteDataE`, desde MEM en el ciclo 16 y desde WB en el 19.
- **Ciclo 22:** `beq x13, x14, ok` recibe sus dos operandos por *forwarding*, resulta tomado y activa `FlushD`/`FlushE`, que anulan `addi x15, x0, -1`.

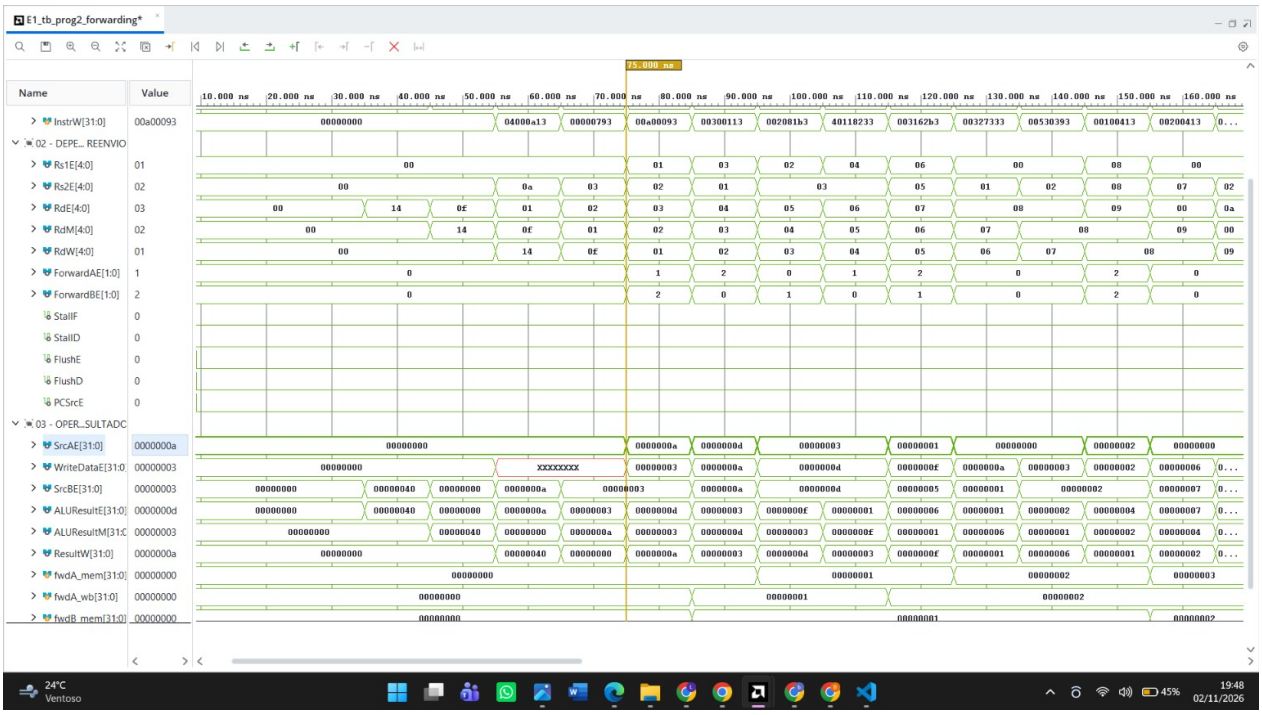


Figura 4: Waveform de P2, toma de datos del ciclo 5

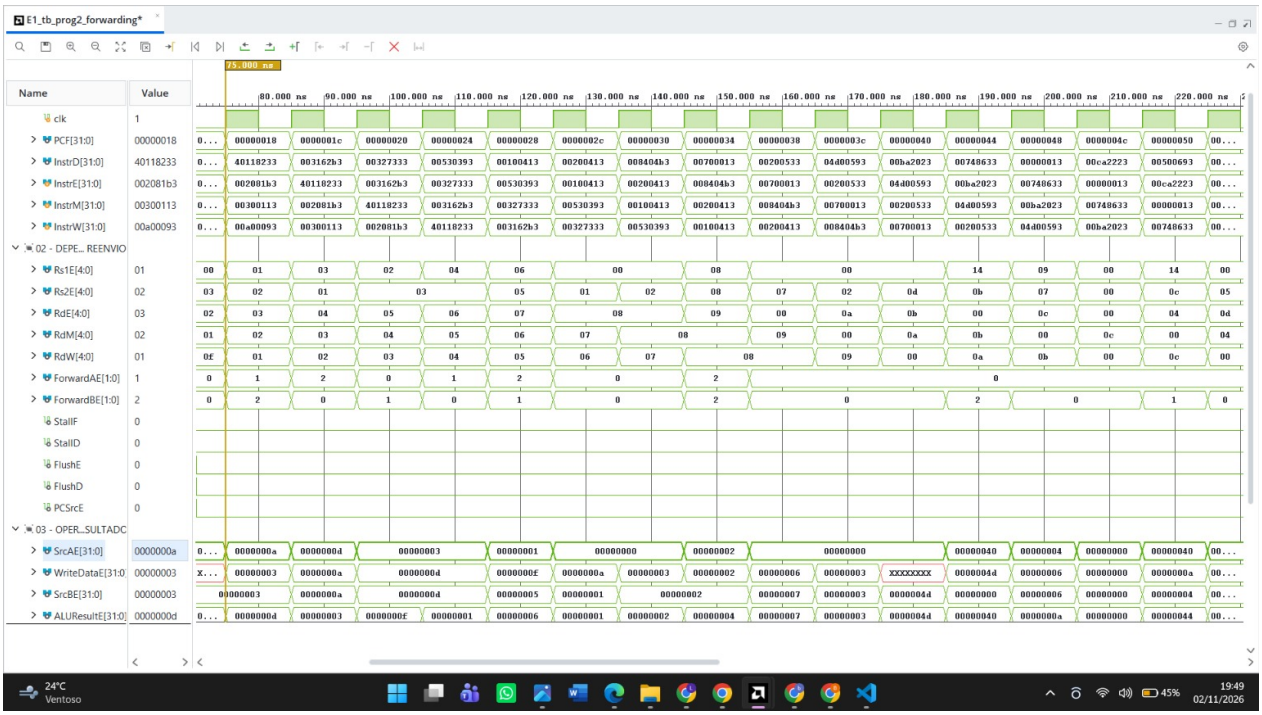


Figura 5: Waveform de P2, ciclos 5 a 22 (80 a 220 ns): `Rs1E`, `Rs2E`, `RdM`, `RdW`, `ForwardAE`, `ForwardBE`, `ALUResultE`, `SrcBE`, `SrcAE` y `WriteDataE`.

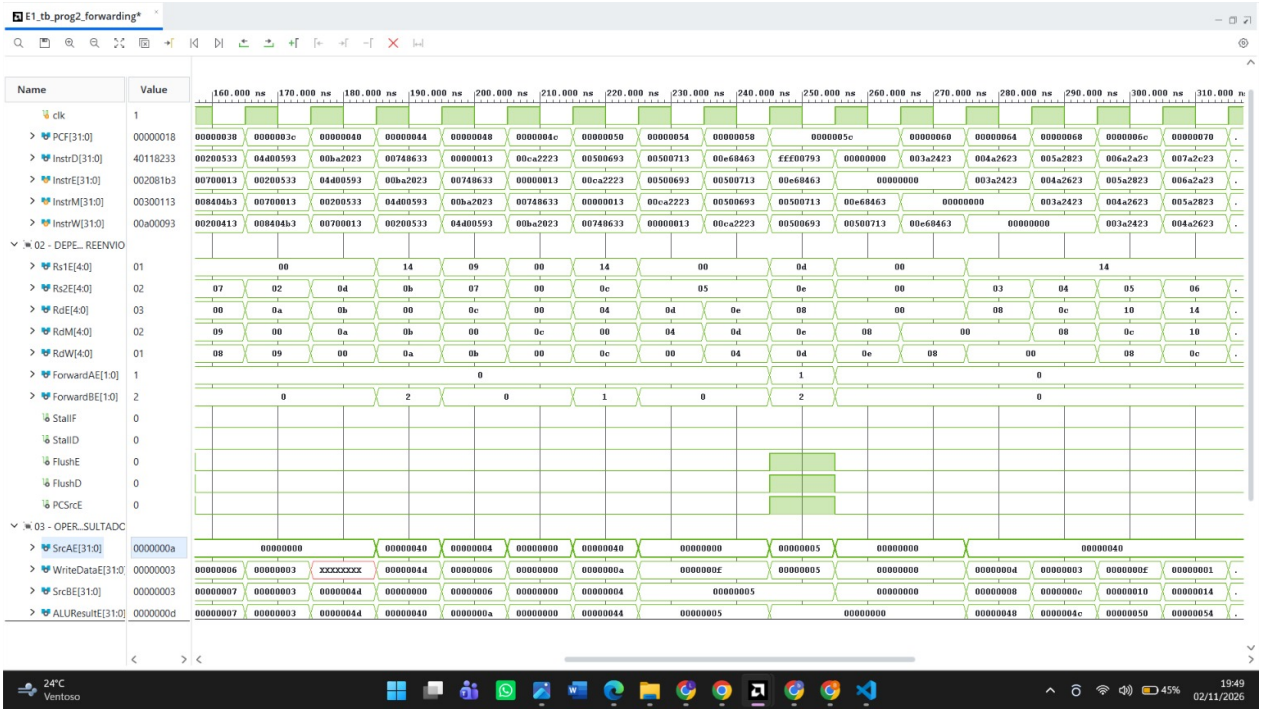


Figura 6: Waveform de P2, ciclos 13 a 28 (160 a 310 ns): forwarding hacia WriteDataE en sw y hacia los operandos del beq tomado en el ciclo 22.

5.3. Programa 3: stalling

Ciclo	t (ns)	IF	ID	EX	MEM	WB	Hazard Unit
6	90	`add x4, x3, x1`	`lw x3, 0(x20)`	`sw x2, 4(x20)`	`sw x1, 0(x20)`	`addi x2, x0, -7`	ForwardBE=01
7	100	`lw x5, 4(x20)`	`add x4, x3, x1`	`lw x3, 0(x20)`	`sw x2, 4(x20)`	`sw x1, 0(x20)`	StallF, StallD, FlushE
8	110	`lw x5, 4(x20)`	`add x4, x3, x1`	`--`	`lw x3, 0(x20)`	`sw x2, 4(x20)`	—
9	120	`sub x6, x1, x5`	`lw x5, 4(x20)`	`add x4, x3, x1`	`--`	`lw x3, 0(x20)`	ForwardAE=01
10	130	`lw x7, 0(x20)`	`sub x6, x1, x5`	`lw x5, 4(x20)`	`add x4, x3, x1`	`--`	StallF, StallD, FlushE
13	160	`lw x8, 4(x20)`	`sw x7, 8(x20)`	`lw x7, 0(x20)`	`sub x6, x1, x5`	`--`	StallF, StallD, FlushE
18	210	`add x10, x0, x1`	`lw x0, 0(x20)`	`add x9, x8, x8`	`nop`	`lw x8, 4(x20)`	ForwardAE=01, ForwardBE=01
19	220	`lw x11, 0(x20)`	`add x10, x0, x1`	`lw x0, 0(x20)`	`add x9, x8, x8`	`nop`	—
21	240	`addi x12, x0, -1`	`beq x11, x1, eq`	`lw x11, 0(x20)`	`add x10, x0, x1`	`lw x0, 0(x20)`	StallF, StallD, FlushE
22	250	`addi x12, x0, -1`	`beq x11, x1, eq`	`--`	`lw x11, 0(x20)`	`add x10, x0, x1`	—
23	260	`lw x13, 4(x20)`	`addi x12, x0, -1`	`beq x11, x1, eq`	`--`	`lw x11, 0(x20)`	ForwardAE=01, FlushD, FlushE
27	300	`sw x4, 12(x20)`	`add x15, x13, x14`	`lw x14, 0(x20)`	`lw x13, 4(x20)`	`--`	StallF, StallD, FlushE

Tabla 7: Diagrama de pipeline de P3 (ciclos seleccionados).

- **Ciclos 7 a 9 (Load-Use en rs1):** con `lw x3` en EX y `add x4, x3, x1` en ID se cumple `ResultSrcE[0] = 1` y `RdE = Rs1D = 3`. `StallF` y `StallD` congelan el PC y el registro IF/ID, de modo que en el ciclo 8 IF e ID repiten sus instrucciones, y `FlushE` inserta la burbuja que aparece en EX. En el ciclo 9 el dato del `lw` ya está en WB y llega por `ForwardAE = 01`.
- **Ciclo 10 (rs2) y ciclo 13 (dato de `sw`):** ocurre lo mismo con `RdE = Rs2D`. El *stall* del ciclo 13 demuestra que un `sw` también debe esperar si su dato proviene de un `lw` inmediatamente anterior.
- **Ciclo 18 (sin *stall*):** `add x9, x8, x8` está a distancia 2 de `lw x8`; el dato ya está en WB y basta con `ForwardAE = ForwardBE = 01`.
- **Ciclo 19 (`lw x0`):** con `lw x0` en EX y `add x10, x0, x1` en ID no hay *stall*, porque `RdE = 0`.
- **Ciclos 21 a 23 (Load-Use en `beq` y *flush*):** primero hay un *stall* por `x11`; luego `beq` recibe `x11` desde WB, se resuelve tomado y anula `addi x12, x0, -1`. Este caso combina los tres mecanismos.
- **Ciclo 27:** quinto *stall*, por `x14`. P3 tiene exactamente 5 ciclos de *stall*.

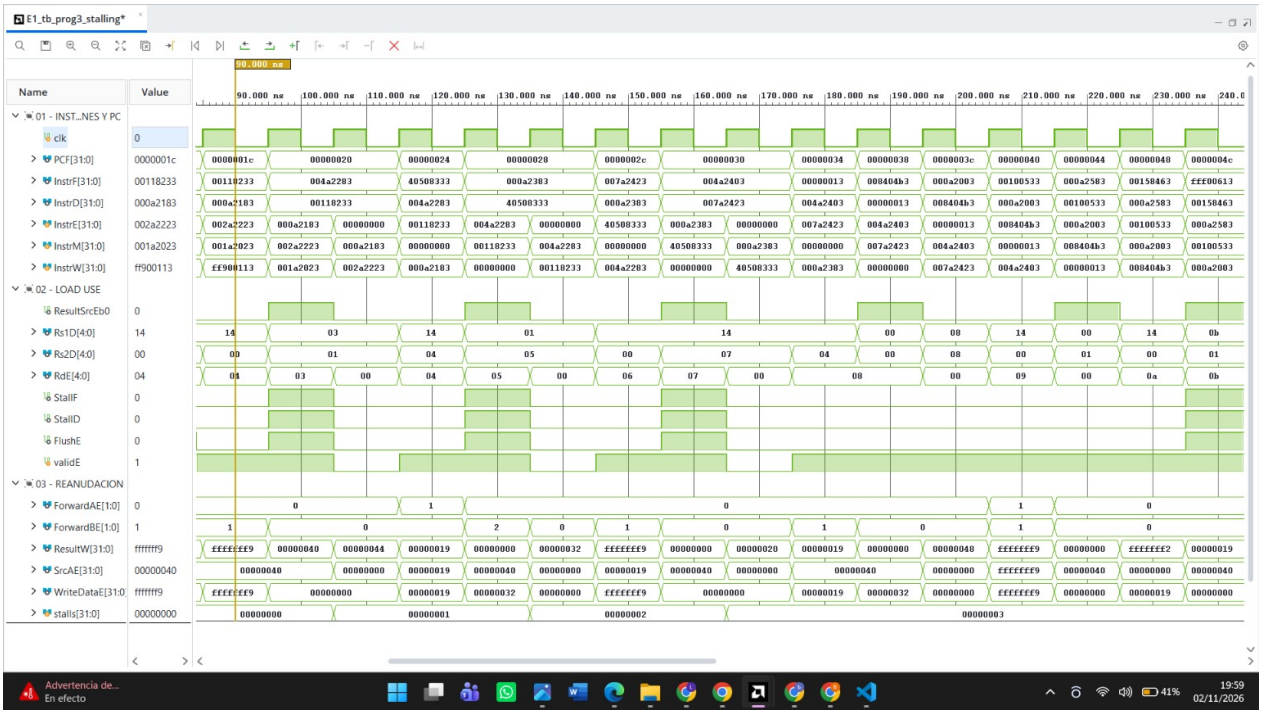


Figura 7: Waveform de P3, ciclos 6 a 10 (90 a 130 ns): `PCF`, `InstrD`, `RdE`, `Rs1D`, `Rs2D`, `ResultSrcE`, `StallF`, `StallD` y `FlushE`.

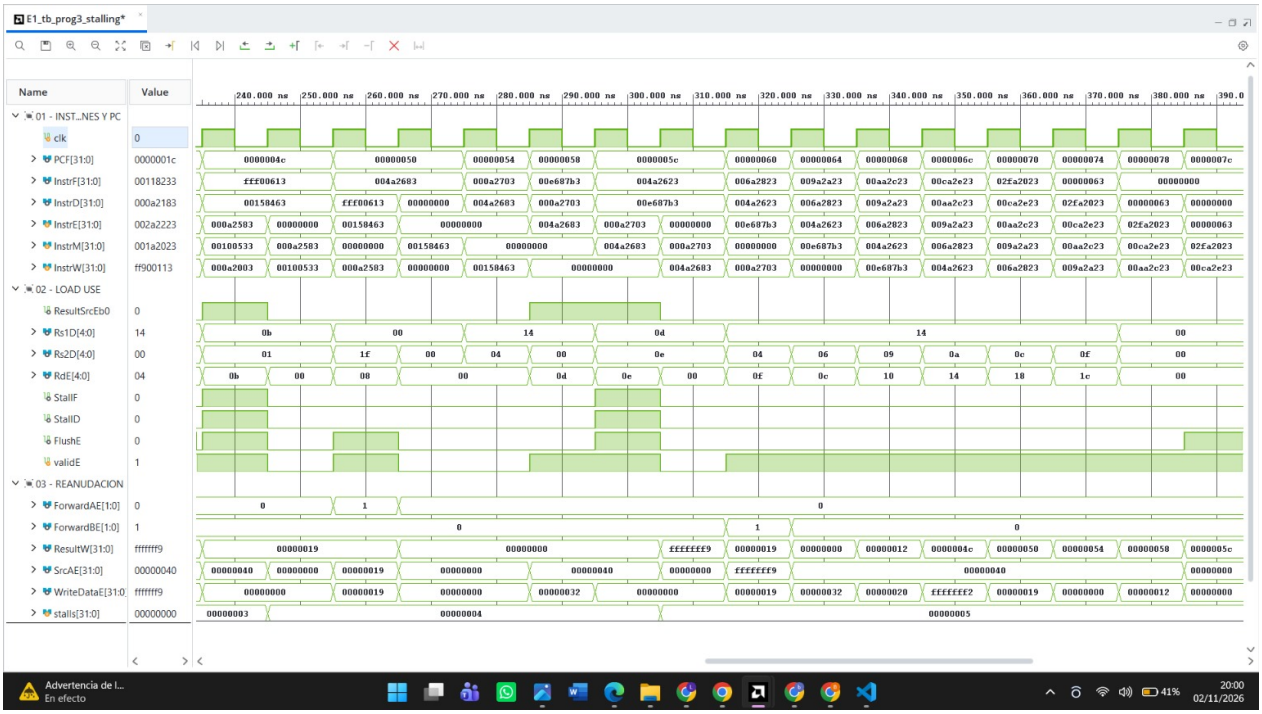


Figura 8: Waveform de P3, ciclos 18 a 23 (210 a 260 ns): ausencia de *stall* con `lw x0` y secuencia *stall* + *flush* del `beq`.

5.4. Programa 4: flushing

Ciclo	t (ns)	IF	ID	EX	MEM	WB	Hazard Unit
5	80	`addi x3, x3, 200`	`addi x3, x3, 100`	`beq x1, x2, t1`	`addi x3, x0, 0`	`addi x2, x0, 4`	ForwardBE=01, FlushD, FlushE
6	90	`addi x3, x3, 1`	`--`	`--`	`beq x1, x2, t1`	`addi x3, x0, 0`	—
9	120	`jal x5, fwd`	`addi x3, x3, 2`	`beq x1, x0, nt`	`addi x3, x3, 1`	`--`	—
11	140	`addi x3, x3, 600`	`addi x3, x3, 500`	`jal x5, fwd`	`addi x3, x3, 2`	`beq x1, x0, nt`	FlushD, FlushE
12	150	`addi x6, x0, 3`	`--`	`--`	`jal x5, fwd`	`addi x3, x3, 2`	—
18	210	`addi x7, x7, 1000`	`jal x0, loop`	`addi x6, x6, -1`	`add x7, x7, x6`	`beq x6, x0, end`	—
19	220	`sw x3, 0(x20)`	`addi x7, x7, 1000`	`jal x0, loop`	`addi x6, x6, -1`	`add x7, x7, x6`	FlushD, FlushE
20	230	`beq x6, x0, end`	`--`	`--`	`jal x0, loop`	`addi x6, x6, -1`	—
34	370	`addi x6, x6, -1`	`add x7, x7, x6`	`beq x6, x0, end`	`--`	`--`	FlushD, FlushE

Tabla 8: Diagrama de pipeline de P4 (ciclos seleccionados).

- **Ciclo 5 (`beq` tomado):** el `beq` llega a EX con `ZeroE = 1`, de modo que `PCSrcE = 1`. Las dos instrucciones del camino incorrecto (`addi x3, x3, 100` en ID y `addi x3, x3, 200` en IF) se descartan: en el ciclo 6, ID y EX contienen burbujas e IF ya busca `t1` (`PCTargetE = 0x1C`).
- **Ciclo 9 (`beq` no tomado):** `beq x1, x0, nt` pasa por EX sin activar `FlushD` ni `FlushE`, y `addi x3, x3, 2` se ejecuta normalmente.
- **Ciclo 11 (`jal`):** `jal` siempre es tomado (`JumpE = 1`); anula `addi x3, x3, 500` y `addi x3, x3, 600`, salta a `fwd` (`PCTargetE = 0x34`) y escribe `x5 = PC + 4 = 0x2C` al llegar a WB.
- **Ciclos 19 y 20 (bucle):** `jal x0, loop` anula `addi x7, x7, 1000` y `sw x3, 0(x20)` (buscada por adelantado) y vuelve a `loop` (`PCTargetE = 0x3C`). Esto se repite en las tres iteraciones: ciclos 19, 25 y 31.
- **Ciclo 34:** con `x6 = 0`, `beq x6, x0, end` es tomado y sale del bucle.

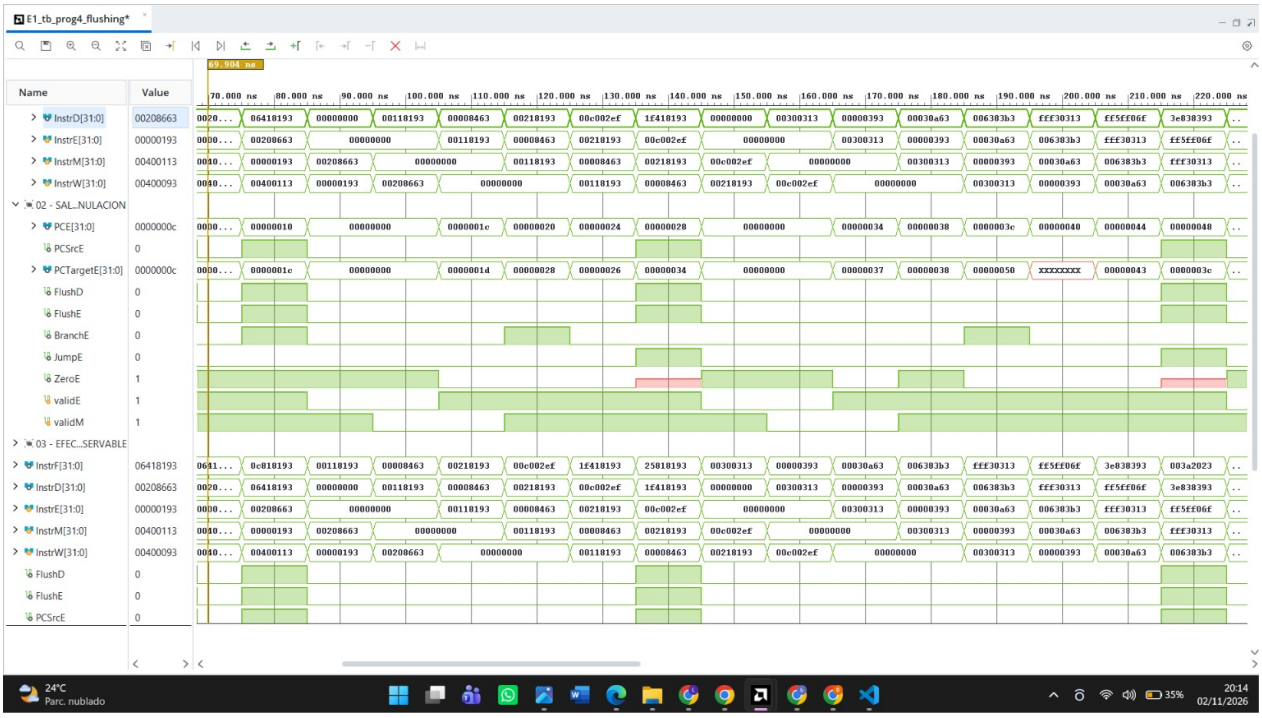


Figura 9: Waveform de P4, ciclos 4 a 12 (70 a 150 ns): `PCF`, `InstrD`, `BranchE`, `JumpE`, `ZeroE`, `PCSrcE`, `PCTargetE`, `FlushD` y `FlushE`.

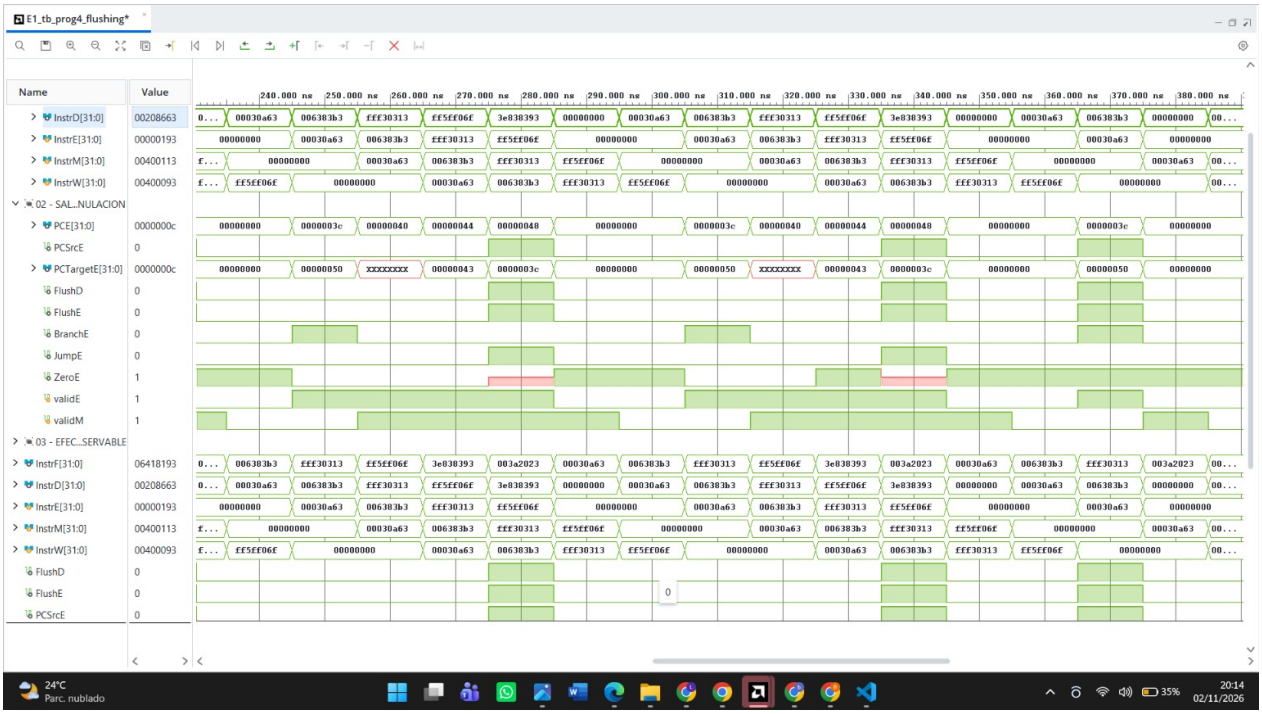


Figura 10: Waveform de P4: iteraciones del bucle y salida en el ciclo 34, ciclos 21 a 35 (240 a 380 ns).

6. Extensión Dual-Issue: Fetch, Decode y datapath preliminar

El procesador ya busca y decodifica dos instrucciones consecutivas por ciclo, y la segunda vía de ejecución está construida hasta WB. Como E1 no exige ejecución simultánea, la Issue Unit preliminar emite solo la vía 1 y el comportamiento es idéntico al Single-Issue: los cuatro programas producen los mismos resultados y los mismos ciclos que antes de la extensión.

6.1. Cambios en Fetch

- **Memoria de instrucciones de doble lectura.** `imem` entrega `rd0 = M[PC]` (vía 1) y `rd1 = M[PC+4]` (vía 2) en el mismo ciclo. Se eligieron dos puertos combinacionales en lugar de una palabra de 64 bits, porque esta última obligaría a alinear el PC a 8 bytes y complicaría los saltos a direcciones impares de palabra. La memoria se inicializa en `0x00000000`, que es una burbuja válida, y recibe el programa mediante el parámetro `MEM_FILE`.
- **Siguiente PC.** Un sumador `PC+8` y el multiplexor `pcseqmux` permiten avanzar una instrucción (+4) o un par completo (+8) según `PCInc8F`. Un salto tomado mantiene la prioridad (`pcmux` con `PCSrcE`). En E1, `PCInc8F = 0`.
- **Registro IF/ID ampliado.** Almacena el par: `InstrD`, `PCD` y `PCPlus4D` para la vía 1, e `InstrD2`, `PCD2 = PC+4` y `PCPlus4D2 = PC+8` para la vía 2. Ambas mitades comparten `StallD` y `FlushD`, por lo que el par se congela o se anula como una unidad.

6.2. Cambios en Decode

- **Segunda unidad de control** (`controller c2`): decodifica `InstrD2` en paralelo con `c`. Cada vía tiene sus propias señales `RegWrite`, `ALUSrc`, `ALUControl`, `ImmSrc`, etc.
- **Segundo extensor de inmediatos** (`ext2`).
- **Register File de 4 lecturas y 2 escrituras.** Los puertos `a1`/`a2` atienden a la vía 1 y `a3`/`a4` a la vía 2. Los dos puertos de escritura operan en flanco de bajada; si ambos escriben el mismo registro, prevalece la vía 2 por ser la instrucción más joven. Las escrituras a `x0` se descartan.

- **Filtro de emisión.** `RegWrite`, `MemWrite`, `Jump` y `Branch` de la vía 2 se combinan con `IssueSlot2D` antes del registro ID/EX; si la vía 2 no se emite, entra a EX como burbuja.

6.3. Módulos incorporados

Módulo / instancia	**Archivo**	**Etapa**	**Función**
`imem` (2 puertos)	`imem.v`	IF	Lectura de `PC` y `PC+4`
`pcadd8`, `pcseqmux`	`datapath.v`	IF	Avance +4 / +8
`instrd2_reg`, `pcd2_reg`, `pcplus4d2_reg`	`datapath.v`	IF/ID	Instrucción y PC de la vía 2
`controller c2`	`riscvpipe.v`	ID	Dual Decode
`ext2`	`datapath.v`	ID	Inmediato de la vía 2
`regfile` 4R/2W	`regfile.v`	ID / WB	Lecturas y escrituras de ambas vías
`issue_unit`	`issue_unit.v`	ID	Genera `IssueSlot2D` y `PCInc8F` (en 0 en E1)
15 registros `*_e2_reg`	`datapath.v`	ID/EX	Control y datos de la vía 2
`srcbmux2`, `alu_unit2`	`datapath.v`	EX	Segunda ALU
`*_m2_reg`, `*_w2_reg`	`datapath.v`	EX/MEM, MEM/WB	Resultado y destino de la vía 2
`resultmux2`	`datapath.v`	WB	Dato para el segundo puerto de escritura

Tabla 9: Módulos incorporados para la arquitectura Dual-Issue.

La memoria de datos y la resolución de saltos siguen siendo únicas, como exige el enunciado. Su asignación dinámica a cualquiera de las dos vías, el *forwarding* de la vía 2 y la lógica de la Issue Unit corresponden a la entrega E2.

6.4. Datapath Dual-Issue preliminar

La Figura 11 presenta el datapath preliminar. La vía 1 coincide con el datapath Single-Issue; la vía 2 se dibuja debajo, con su propia unidad de control (señales agrupadas), el filtro `IssueSlot2D`, la segunda ALU y el segundo puerto de escritura del banco de registros, que es un único banco 4R/2W representado con sus puertos de cada vía.

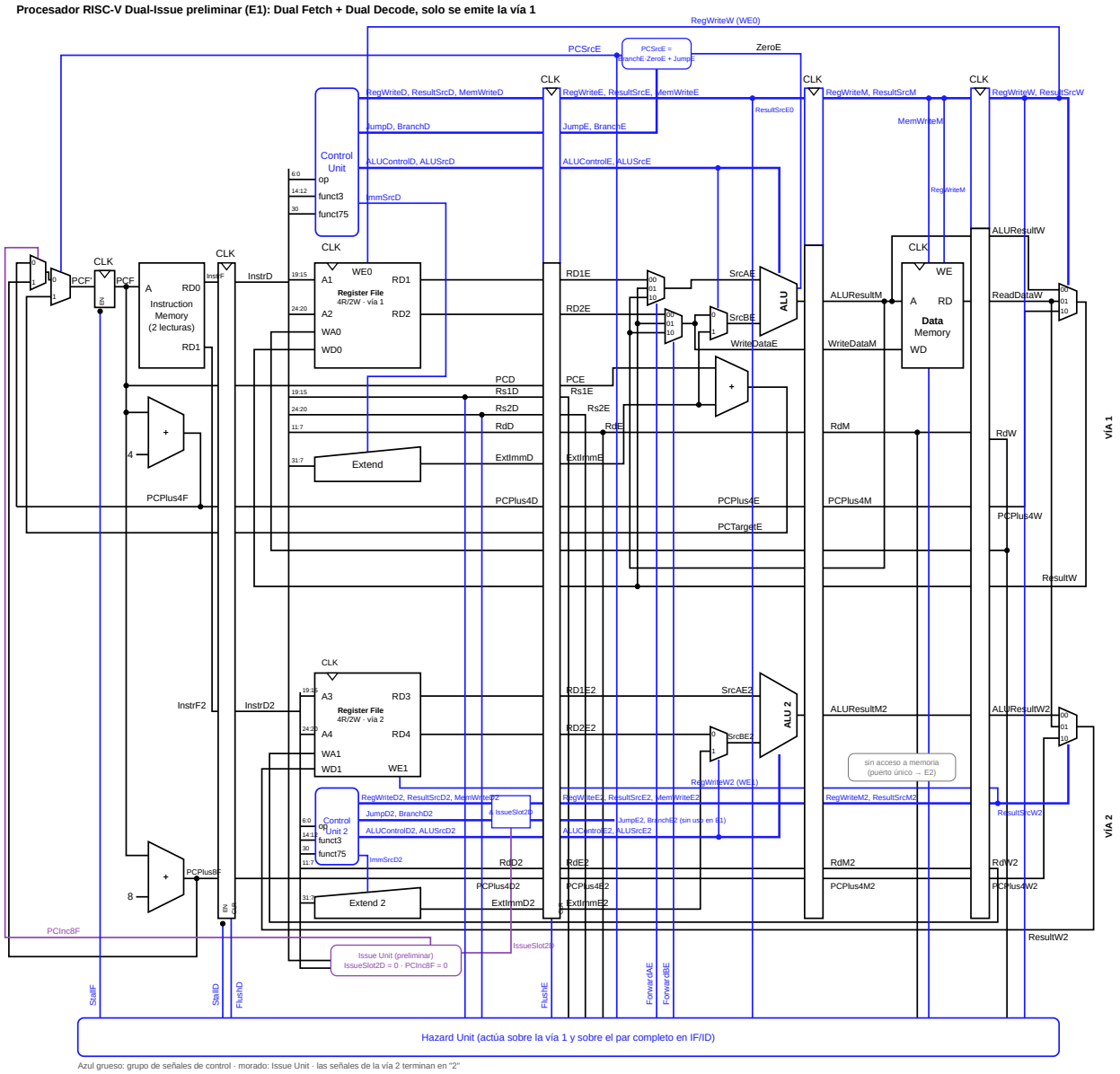


Figura 11: Datapath Dual-Issue preliminar (E1): Dual Fetch y Dual Decode; solo se emite la vía 1.

6.5. Evidencia de Dual Fetch y Dual Decode

La Tabla 10 proviene de la simulación y muestra el par que ocupa ID en los primeros ciclos de P1.

Prog.	Ciclo	t (ns)	PCD	InstrD	PCD2	InstrD2	Evento
P1	0	30	0x00	00c00093 (addi x1)	0x04	00500113 (addi x2)	—
P1	1	40	0x04	00500113 (addi x2)	0x08	ffd00193 (addi x3)	—
P1	2	50	0x08	ffd00193 (addi x3)	0x0C	0f000213 (addi x4)	—

Tabla 10: Contenido del registro IF/ID (par de instrucciones).

- **Dual Fetch:** en todos los ciclos se cumple `PCD2 = PCD + 4` e `InstrD2` es la palabra siguiente a `InstrD`.
- **Dual Decode:** `c2` produce señales válidas para `InstrD2`; por ejemplo, en el ciclo 0 de P1, `RegWriteD2 = 1` y `ALUSrcD2 = 1` (es un `addi`).

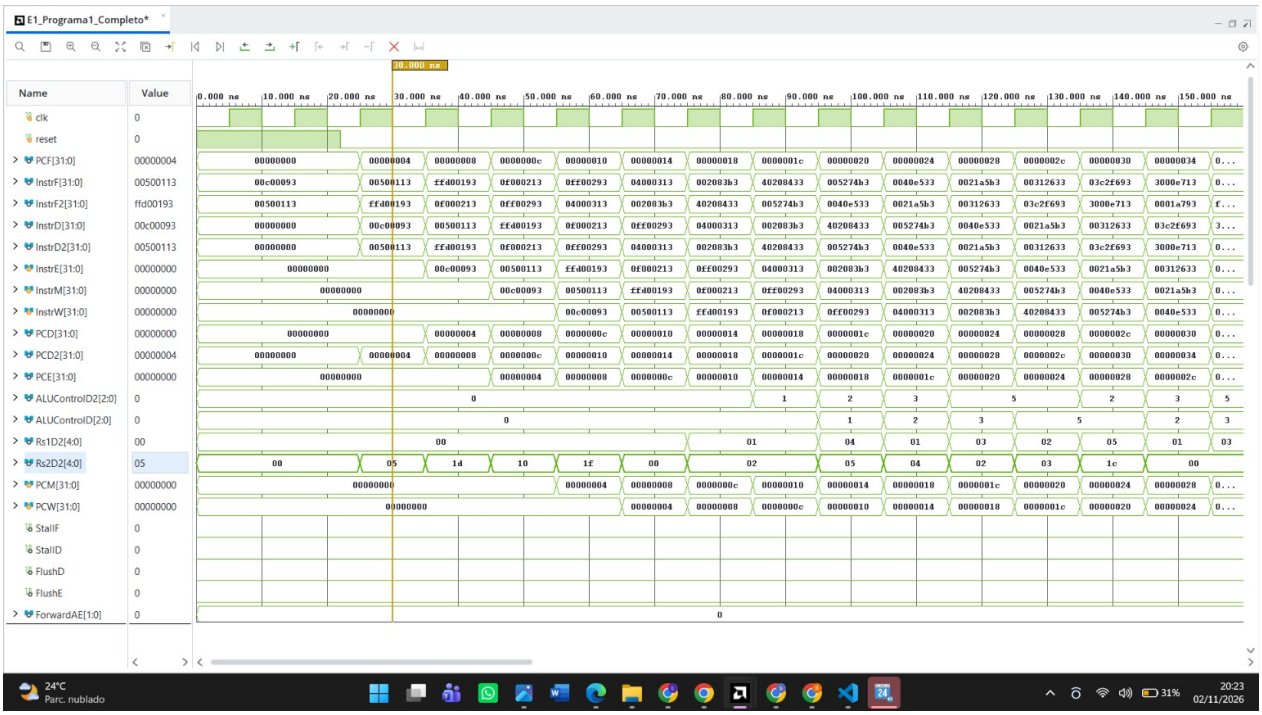


Figura 12: Waveform de P1, ciclos 0 a 5 (30 a 80 ns): `PCF`, `Instr`, `Instr2`, `PCD`, `InstrD`, `PCD2`, `InstrD2`, `ALUControlD2`.

7. Conclusiones

- El pipeline RV32I de cinco etapas funciona correctamente con *forwarding*, *stalling* y *flushing*. La corrección de la burbuja en `maindec` fue indispensable: sin ella, el primer salto tomado propagaba `x` hasta el PC. Los cuatro programas de prueba y el test de Harris & Harris producen los resultados esperados.
- La base Dual-Issue está lista: Dual Fetch, Dual Decode, Register File 4R/2W, segunda ALU y registros de pipeline de la vía 2. Con la vía 2 deshabilitada, el procesador es equivalente ciclo a ciclo al Single-Issue.
- Para E2 quedan pendientes la lógica de la Issue Unit, el *forwarding* entre vías, la asignación dinámica del puerto de memoria y de la unidad de saltos, y el *flush* selectivo cuando el salto ocupa la primera posición del par.